

Wheat Disease Forecasting Using Elastic Net Regression

Author: Alex Rabeau

Date: March 2026

Introduction

This report presents a forecasting pipeline for predicting wheat disease severity and crop incidence across UK regions using an Elastic Net regression framework.

The workflow integrates multiple datasets, including pest observations, agronomic variables, bioclimatic indicators, fungicide usage, and land-use composition.

Key features include lagged variable construction, Kalman-based imputation, skewness correction, and region-specific modelling.

The target variables are:

- **L1_Disease_Severity**
 - **L1_Crop_Incidence**
 - **L2_Disease_Severity**
 - **L2_Crop_Incidence**
-

1. Data Loading and Harmonisation

We begin by extracting and cleaning disease observations from multiple Excel sheets.

Column names are standardised and unnecessary variables removed to ensure consistency across sources.

```
import pandas as pd
import numpy as np

PEST_FILE = "data/regional-mean-time-series-wheat-october-2025.xlsx"

def process_leaf(sheet, prefix):

    df = pd.read_excel(PEST_FILE, sheet_name=sheet)

    df.columns = df.columns + df.iloc[0].astype(str)
    df = df.iloc[1:]

    df = df.loc[:, df.columns.str.contains(
        "Survey_year|Region|Zymoseptoria_tritici"
    )]

    df.columns = df.columns.str.replace(".*Survey_year.*", "Survey_year", regex=True)
    df.columns = df.columns.str.replace(".*Region.*", "Region", regex=True)
    df.columns = df.columns.str.replace(".*severity.*", f"{prefix}_Disease_Severity", regex=True)
    df.columns = df.columns.str.replace(".*Plant Incidence.*", f"{prefix}_Plant_Incidence", regex=True)
    df.columns = df.columns.str.replace(".*Crop Incidence.*", f"{prefix}_Crop_Incidence", regex=True)

    return df
```

```

leaf1 = process_leaf("Leaf 1 (Flag leaf)", "L1")
leaf2 = process_leaf("Leaf 2", "L2")

regional_wheat_leaf = pd.merge(
    leaf1,
    leaf2,
    on=["Survey_year", "Region"]
)

regional_wheat_leaf = regional_wheat_leaf[
    regional_wheat_leaf["Region"] != "Unknown"
].drop(columns=["L1_Plant_Incidence", "L2_Plant_Incidence"])

```

2. Feature Integration

The core dataset is enriched by merging with agronomic, climate, fungicide, and land-use datasets.

All variables are aligned by Year and Region.

```

AGRONOMIC_FILE = "data/agronomic_data.csv"
BIOCLIM_FILE = "data/bioclim_data.csv"
FUNGICIDE_FILE = "data/fungicide_data.csv"
LUC_FILE = "data/prop_LUC.csv"

agronomic_data = pd.read_csv(AGRONOMIC_FILE)
bioclim_data = pd.read_csv(BIOCLIM_FILE)
fungicide_data = pd.read_csv(FUNGICIDE_FILE)
luc_data = pd.read_csv(LUC_FILE)

FORECASTING_DF = regional_wheat_leaf.rename(columns={"Survey_year": "Year"})
FORECASTING_DF["Year"] = FORECASTING_DF["Year"].astype(int)

FORECASTING_DF = (
    FORECASTING_DF
    .merge(agronomic_data, on=["Year", "Region"], how="left")
    .merge(bioclim_data, on=["Year", "Region"], how="left")
    .merge(fungicide_data, on=["Year", "Region"], how="left")
    .merge(luc_data, on=["Year", "Region"], how="left")
)

```

3. Lagged Feature Engineering

To capture temporal dynamics, we introduce lagged versions of all target variables up to 3 years within each region.

```

TARGET_VARS = [
    "L1_Disease_Severity",
    "L1_Crop_Incidence",
    "L2_Disease_Severity",
    "L2_Crop_Incidence"
]

```

```

N_LAGS = 3

lagged = FORECASTING_DF.copy()
lagged = lagged.sort_values(["Region", "Year"])

for col in TARGET_VARS:
    for lag in range(1, N_LAGS+1):
        lagged[f"{col}_lag{lag}"] = (
            lagged.groupby("Region")[col].shift(lag)
        )

lagged = lagged.drop(columns=TARGET_VARS)

FORECASTING_DF = FORECASTING_DF.merge(
    lagged,
    on=["Year", "Region"],
    how="left"
)

```

4. Missing Data Imputation (Kalman Filtering)

Missing values in predictor variables are imputed using a Kalman smoothing approach, which preserves temporal structure.

```

from pykalman import KalmanFilter

def kalman_impute(series):

    s = series.replace([np.inf, -np.inf], np.nan)

    if s.notna().sum() < 3:
        return s

    filled = s.ffill().bfill()
    values = filled.values
    mask = np.isnan(s.values)

    kf = KalmanFilter(initial_state_mean=values[0])

    try:
        smoothed, _ = kf.em(values, n_iter=5).smooth(values)
        smoothed = smoothed.flatten()
        values[mask] = smoothed[mask]
    except:
        values[mask] = np.nanmean(values)

    return pd.Series(values, index=s.index)

disease_cols = [c for c in FORECASTING_DF.columns if "L1" in c or "L2" in c]

for col in FORECASTING_DF.columns:
    if col not in ["Year", "Region"] and col not in disease_cols:

```

```
FORECASTING_DF[col] = kalman_impute(FORECASTING_DF[col])

FORECASTING_DF = FORECASTING_DF.dropna()
```

5. Handling Skewness

Predictor variables exhibiting high skewness ($|\text{skew}| > 1$) are transformed to stabilise variance and improve model performance.

```
from scipy.stats import skew

predictors = [
    c for c in FORECASTING_DF.columns
    if c not in ["Year", "Region"] + TARGET_VARS
]

skewness_results = []

for col in predictors:

    x = FORECASTING_DF[col]
    skew_val = skew(x, nan_policy="omit")
    transformation = "none"

    if abs(skew_val) > 1:

        if skew_val > 1 and (x > 0).all():
            FORECASTING_DF[col] = np.log1p(x)
            transformation = "log1p"

        elif skew_val > 1:
            FORECASTING_DF[col] = np.sqrt(x - x.min() + 1)
            transformation = "sqrt"

        elif skew_val < -1:
            FORECASTING_DF[col] = x**2
            transformation = "squared"

    skewness_results.append({
        "variable": col,
        "skewness": skew_val,
        "transformation": transformation
    })

skewness_results = pd.DataFrame(skewness_results)
```

6. Region-Specific Elastic Net Modelling

Models are trained separately for each region using an Elastic Net regression with cross-validated regularisation.

- Training period: up to 2020

- Testing period: 2021–2025
- Forecast: 2026

```

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import ElasticNetCV, ElasticNet
from sklearn.metrics import mean_squared_error

TRAIN_END = 2020
TEST_END = 2025

forecast_results = []
performance_results = []

regions = FORECASTING_DF["Region"].unique()

for reg in regions:

    df_region = FORECASTING_DF[FORECASTING_DF["Region"]==reg]

    train = df_region[df_region["Year"]<=TRAIN_END]
    test = df_region[(df_region["Year"]>TRAIN_END) & (df_region["Year"]<=TEST_END)]

    scaler = StandardScaler()
    X_train = scaler.fit_transform(train[predictors])
    X_test = scaler.transform(test[predictors])

    for target in TARGET_VARS:

        y_train = train[target]
        y_test = test[target]

        enet_cv = ElasticNetCV(l1_ratio=0.5, cv=5, random_state=123)
        enet_cv.fit(X_train, y_train)

        model = ElasticNet(alpha=enet_cv.alpha_, l1_ratio=0.5)
        model.fit(X_train, y_train)

        pred_test = model.predict(X_test)

        rmse = np.sqrt(mean_squared_error(y_test, pred_test))

        performance_results.append({
            "region": reg,
            "target": target,
            "rmse": rmse,
            "min": df_region[target].min(),
            "max": df_region[target].max()
        })

    X_future = df_region[df_region["Year"]==TEST_END][predictors]

    if not X_future.empty:

        X_future = scaler.transform(X_future)

```

```

pred_2026 = model.predict(X_future)[0]

forecast_results.append({
    "region": reg,
    "target": target,
    "year": 2026,
    "forecast_value": float(pred_2026)
})

```

7. Model Evaluation and Forecast Output

Model performance is evaluated using RMSE and normalised relative RMSE to allow comparison across regions and targets.

```

performance_results = pd.DataFrame(performance_results)

performance_results["relative_rmse_pct"] = (
    performance_results["rmse"] /
    (performance_results["max"] - performance_results["min"]))
) * 100

forecast_results = pd.DataFrame(forecast_results)

forecast_results.to_csv("pest_forecasts_2026.csv", index=False)
performance_results.to_csv("pest_model_performance.csv", index=False)

```

Conclusion

This pipeline demonstrates a robust and scalable approach for forecasting wheat disease dynamics using integrated environmental and agronomic data.

Key strengths include:

- Region-specific modelling to capture spatial patterns
- Lagged features to account for temporal dependence
- Kalman imputation to handle data gaps
- Elastic Net regularisation for stability in high-dimensional settings

The resulting forecasts provide actionable insights for agricultural planning and disease management strategies.